# Evaluation Method for Feature Matching and Visual Localization

Feature matching algorithm is an important part of the final system. The quality predicted correspondences between satellite and UAV images controls the accuracy of final location prediction. However, there is no consensus on what constitute a good match without labor-intensive ground-truth labeling process. Therefore, downstream tasks, e.g pose estimation, scene reconstruction, are often used to measure the goodness of matches.

## Dataset

### UAV Captured Video Stream

Each video frame is accompanied by geographic and operational metadata (e.g location, altitude, heading, speed). This includes the estimated position (lon, lat) of each frame's corner. The video stream can be temporally down-sampled to create a dataset of evenly timed frames with metadata for faster evaluation process (approximation to real performance).

### Satellite Images

For waypoint-based visual localization, each landmark's images are grouped into one folder. The images are different versions of the landmark with different viewpoints and scales for better retrieval. Each image also comes with metadata about corners' position. This can be used to calculate the position of every pixel within the image. The current organization and metadata of images are not optimized and likely will change in future iteration.

## Performance metrics

In this section, we establish 3 important metrics for evaluation of feature matching methods, how to calculate them and visualization of those results.

# Retrieval Performance

Accuracy, Precision, Recall, FPR. @5m

The location of the centered pixel will be regarded as ground truth for the retrieval system, also the target for our prediction. Because it is unlikely that we uncover the absolute correct location, we will define performance with respect to a threshold (e.g @5m or @10m).

## Nomenclature

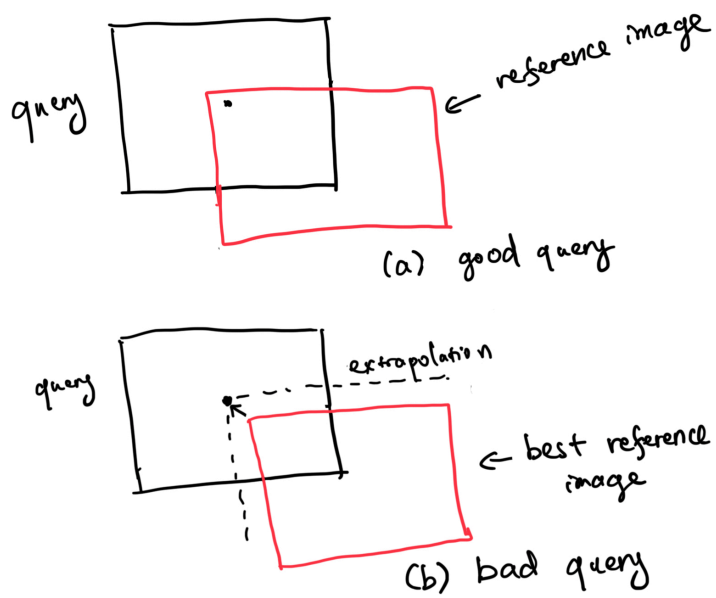


Illustration of good and bad query

There will be 2 types of queries made to the system: image with/without reference database images. We need to differentiate these types as they give rise to different kind of errors. Hereby, we will call them good query and bad query, respectively. How do we label them as such? For now, we decided that good query will have their center pixel's location lie within the area of any reference images. This may sound too strict as the system are capable of deduce the query image center pixel from partially overlap reference images. However, this require

extrapolation from the calculated homography (again trade off of expressiveness and correctness) and complicated the metrics computation. For the good query, the prediction will be classified as consistent or inconsistent. There will be 5 cases for a query result:

1. True Positive: the query is good and the prediction consistent.
2. False Positive 1: the query is good and the prediction inconsistent.
3. False Positive 2: the query is bad and there is a prediction.
4. False Negative: the query is good and there is no prediction.
5. True Negative: the query is bad and there is no prediction.

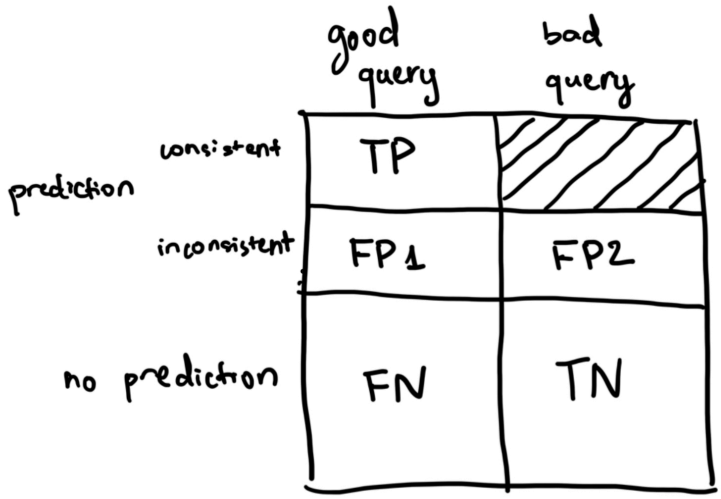


Illustration of different query result

In position calibration, making a false prediction about current location is more disastrous than doing nothing (what if we bomb the wrong place, like our beloved GS's home instead of DT's for example 🙂). We care much more about minimizing error (punish false overconfidence) than occasionally reserving our prediction, which is a weird way to say that we're risk adverse. We need the system to have (1) high precision and (2) high recall (as a secondary priority). **We need to decide whether to discriminate FP1 and FP2. Any other metrics?**

## Matching Accuracy

Given the dataset's detail, we can make more fine-grained evaluation at pixel level. Because we know the locations of image's corner, we can calculate the ground-truth locations of detected keypoints in UAV captured image and their assigned locations inferred from their correspondences with satellite images.

We can visualize the histogram of errors (measured in metre) in a single images and across all successful retrieval. However, the definition of "successful retrieval" depend directly on the threshold on the number of inlier keypoints and strictness of RANSAC, we need to tune those value before this visualization step. Hopefully, this will reveal the relationship between match scores, RANSAC parameters (confidence, iteration, e.t.c) and and prediction accuracy (which heavily depends on accuracy measurement and position estimation, or "we cannot take the input as given").

## Timing / Computation

Because our solution will be deployed real time on edge device, we need to respect the problem and system constraints. There are strict requirements on timing but less so on computation. The timing should also provide an accurate view of the time requirement of the whole pipeline (profiling), so we can optimize and improve its efficiency. There are several steps within a typical feature matching pipeline.

| Step | Responsibility | Characteristic |
|---|---|---|
| Pre-processing | Convert raw data to suitable format for consumption. This includes loading, converting to gray scale, enhancement and transformation (rotating, scaling, contrast boosting, e.t.c) | Most implementation is in CPU. If this becomes the bottleneck, we can just move to GPU to do this. |
| Keypoint Detection + Description | As the name suggests | On GPU **(Do we need synchronization to reliably measure GPU time and CPU time? What about host2device transfer time?)** |
| Keypoint Matching | As the name suggests | On GPU |

| | | |
|---|---|---|
| Post-processing | Filter the obtained matches to ensure consistency with respect to the dominated homography (remove outliers) | There are both implementation on GPU and CPU **(but to my knowledge CPU version is faster, maybe because of sync lock and data transfer. GPU version is implemented by kornia library, I need to profile this also and check this again)** |

We also need to report total processing time for matching of a single pair and localization of a single query. If we report this total time directly then it will be bigger than actual processing time as there are overhead of timing measurement.

# Experiment and Evaluation Design

# Pitfalls

Common pitfalls that may trip off the performance measurement.

1. The accuracy of ground-truth
2. Geographic drift
3.

# Implementation Checklist

We need to decide whether to compute the metric directly or stored the result and measure performance as a separate step. I think I've just made up my mind of doing so in 2 stages. It will be less intrusive to the inner working, less synchronization e.t.c. Why here. How to design those system. If we need to store immediate calculation, how can we structure them to ensure clarity and reproducibility and maintainability of the system? How flexible should the system be? How to run them? How to do it fast and accurate so that we can minimize time on this step? The first iteration through will be slow but it should improve with time

as this will be the singular most important module in the whole system. We should separate 2 stages. Inference and Evaluation. This allows for SoC and compatibility with a variety of inference engine and programming language.

- [ ] Expectation of the evaluation system (Requirement)
- [ ] Dummy baseline (SuperPoint + LightGlue (huggingface))
- [ ] Structure of the intermediate result
- [ ] Implementation Plan and Checklist

# Results

| Method | | | |
|---|---|---|---|
| | | | |
| | | | |